

BÁO CÁO BÀI TẬP LẬP TRÌNH

Phân tích Tính Đa Hình, Kế Thừa & Giao Diện Comparable trong Java

Phần 1: Tính Đa Hình (Polymorphism) và Kế Thừa (Inheritance)

1. What are the advantages of Polymorphism? (Ưu điểm của tính Đa hình là gì?)

Tính đa hình (Polymorphism) giúp chương trình trở nên linh hoạt hơn vì cùng một phương thức có thể hoạt động theo nhiều cách khác nhau tùy vào đối tượng sử dụng. Nhờ đó, lập trình viên có thể tái sử dụng mã nguồn hiệu quả hơn, giảm sự trùng lặp và giúp chương trình dễ bảo trì, dễ mở rộng khi phát triển thêm các lớp mới trong hệ thống.

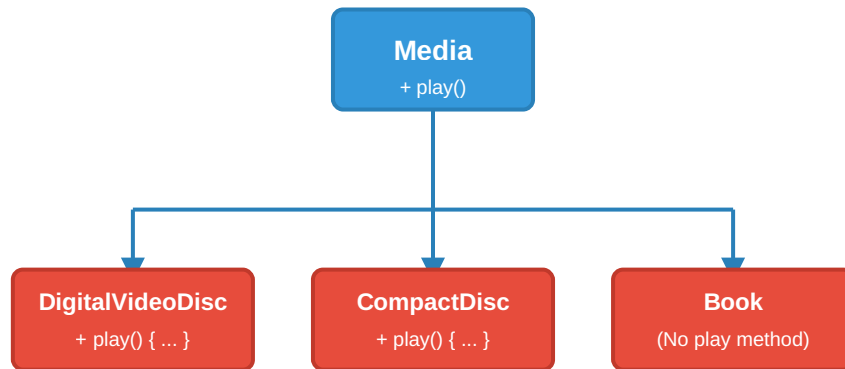
2. How is Inheritance useful to achieve Polymorphism in Java? (Kế thừa giúp đạt được tính Đa hình trong Java như thế nào?)

Kế thừa (Inheritance) cho phép lớp con kế thừa các thuộc tính và phương thức từ lớp cha, đồng thời có thể ghi đè (override) phương thức để tạo ra cách xử lý riêng. Khi sử dụng tham chiếu của lớp cha để quản lý đối tượng lớp con, Java sẽ tự động gọi phương thức phù hợp của lớp con khi chương trình chạy (Dynamic Method Dispatch). Đây chính là cơ chế cốt lõi để kế thừa hỗ trợ thực hiện đa hình.

3. What are the differences between Polymorphism and Inheritance in Java? (Sự khác nhau giữa Đa hình và Kế thừa?)

Kế thừa là cơ chế tạo mối quan hệ cấu trúc giữa các lớp (IS-A relationship), giúp lớp con sử dụng lại những gì lớp cha đã có. Trong khi đó, Đa hình là khả năng về mặt hành vi, cho phép cùng một lời gọi phương thức thể hiện nhiều hành vi khác nhau trên các đối tượng khác nhau. Nói cách khác, kế thừa tạo nền tảng cấu trúc, còn đa hình giúp tận dụng nền tảng đó để quá trình thực thi linh hoạt hơn.

Minh họa Kế Thừa & Đa Hình trong hệ thống AIMS



Cùng gọi `m.play()` nhưng DVD và CD có cách phát nội dung khác nhau

Phần 2: Sử dụng Comparable Interface cho việc sắp xếp

1. Lớp nào nên implement interface Comparable?

Vì chúng ta muốn so sánh các mục (items) trong giỏ hàng (Cart) với nhau, lớp cha Media nên implement interface Comparable<Media>. Điều này đảm bảo rằng tất cả các loại media (như DVD, CD, Book) đều chia sẻ một trật tự sắp xếp tự nhiên cơ bản (ví dụ: theo tên tiêu đề).

2. Trong các lớp đó, làm thế nào để implement phương thức compareTo() để phản ánh đúng thứ tự mong muốn?

Ta cần ghi đè (override) phương thức compareTo(Media other) trong lớp Media. Giả sử ta muốn sắp xếp theo tiêu đề (Title) trước, nếu tiêu đề giống nhau thì xếp theo chi phí (Cost). Cách làm như sau:

```
@Override
public int compareTo(Media other) {
    // 1. So sánh theo Title
    int titleComparison = this.getTitle().compareTo(other.getTitle());
    if (titleComparison != 0) {
        return titleComparison;
    }
    // 2. Nếu Title giống nhau, so sánh theo Cost giảm dần
    return Double.compare(other.getCost(), this.getCost());
}
```

3. Chúng ta có thể có 2 quy tắc sắp xếp (theo title rồi cost VÀ theo cost rồi title) nếu dùng cách tiếp cận Comparable này không?

KHÔNG. Interface Comparable chỉ cho phép định nghĩa **một trật tự tự nhiên duy nhất** (Natural Ordering) cho một lớp thông qua phương thức compareTo(). Nếu muốn hỗ trợ nhiều tiêu chí sắp xếp khác nhau tùy thời điểm (ví dụ: lúc thì sort theo Title, lúc thì sort theo Cost), chúng ta bắt buộc phải tạo các lớp riêng biệt implement interface Comparator.

4. Giả sử DVD có quy tắc sắp xếp khác với các loại Media khác (theo Title -> giảm dần theo Length -> Cost). Làm thế nào để sửa code cho phép điều này?

Ta có thể giải quyết bằng cách ghi đè lại (override) phương thức compareTo() bên trong chính lớp DigitalVideoDisc (lớp con của Media). Khi đối tượng được gọi là DVD, tính đa hình sẽ kích hoạt phương thức compareTo() của DVD thay vì của Media cha.

```

@Override
public int compareTo(Media other) {
    if (other instanceof DigitalVideoDisc) {
        DigitalVideoDisc otherDVD = (DigitalVideoDisc) other;
        int titleCompare =
this.getTitle().compareTo(otherDVD.getTitle());
        if (titleCompare != 0) return titleCompare;

        // Sắp xếp Length giảm dần (đảo ngược vị trí tham số)
        int lengthCompare = Integer.compare(otherDVD.getLength(),
this.getLength());
        if (lengthCompare != 0) return lengthCompare;

        // Sắp xếp Cost
        return Double.compare(this.getCost(), otherDVD.getCost());
    }
    // Chuyển về so sánh mặc định của lớp cha nếu không phải so sánh 2
    DVD
    return super.compareTo(other);
}

```

Khác biệt giữa Comparable và Comparator

Comparable

Cài đặt BÊN TRONG lớp
Chỉ có 1 cách sort duy nhất
obj1.compareTo(obj2)

Comparator

Cài đặt BÊN NGOÀI lớp
Có thể tạo vô số cách sort
compare(obj1, obj2)